

```
import pandas as pd
a=pd.Series([1,2,3,4,5])
print(a)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
!pip install pandas
```

```
Requirement already satisfied: pandas in c:\users\devad\anaconda3\lib\
site-packages (2.2.3)
```

```
Requirement already satisfied: numpy>=1.26.0 in c:\users\devad\
anaconda3\lib\site-packages (from pandas) (2.1.3)
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\
devad\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
```

```
Requirement already satisfied: pytz>=2020.1 in c:\users\devad\
anaconda3\lib\site-packages (from pandas) (2024.1)
```

```
Requirement already satisfied: tzdata>=2022.7 in c:\users\devad\
anaconda3\lib\site-packages (from pandas) (2025.2)
```

```
Requirement already satisfied: six>=1.5 in c:\users\devad\anaconda3\
lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
import pandas as pd
a=pd.Series(["pratho","devashree","monika","tarunika","varshini","sanj
eevi"])
print(a)
print(a.values)
print(a.index)
print(a.dtype)
#accessing elements
print(a[1])
#access element using loc
print(a.loc[1])
print(a.iloc[1])
#slicing
print(a[1:3])
print(a.loc[1:3])
print(a.iloc[2:5])
```

```
0    pratho
1  devashree
2    monika
3  tarunika
4  varshini
5  sanjeevi
dtype: object
```

```

['pratho' 'devashree' 'monika' 'tarunika' 'varshini' 'sanjeevi']
RangeIndex(start=0, stop=6, step=1)
object
devashree
devashree
devashree
1    devashree
2         monika
dtype: object
1    devashree
2         monika
3    tarunika
dtype: object
2         monika
3    tarunika
4    varshini
dtype: object

```

```

#initializing index values
import pandas as pd
arr=pd.Series([1,2,3,4,5,6],index=['A','B','C','D','E','F'])
print(arr)
print(arr["E"])
#vectorized operation - no loop needed
print(arr+5)
print(arr-5)
print(arr.iloc[1]) #can access only by the original position
print(arr.loc["A"]) #can access by the labelled index or original position
#statistical function
print(arr.sum())
print(arr.max())
print(arr.min())
print(arr.std())
print(arr.mean())
print(arr.median())
#boolean indexing
print(arr[arr>2])
print(arr.count())
print(len(arr))
#it return individual value count
print(arr.value_counts())
#string operation
name=pd.Series(["jack","vishnu","anitha","sasmitha","srinivas"])
print(name.str.upper())
print(name.str.len())

```

```

A    1
B    2
C    3

```

```
D      4
E      5
F      6
dtype: int64
5
A      6
B      7
C      8
D      9
E     10
F     11
dtype: int64
A     -4
B     -3
C     -2
D     -1
E      0
F      1
dtype: int64
2
1
21
6
1
1.8708286933869707
3.5
3.5
C      3
D      4
E      5
F      6
dtype: int64
6
6
1      1
2      1
3      1
4      1
5      1
6      1
Name: count, dtype: int64
0      JACK
1      VISHNU
2      ANITHA
3      SASMITHA
4      SRINIVAS
dtype: object
0      4
1      6
```

```

2      6
3      8
4      8
dtype: int64

import pandas as pd
a=pd.Series({
    "gayathri":16,
    "rithanya":38,
    "bavanthika":8})
print(a)
print(a["rithanya"])

gayathri      16
rithanya      38
bavanthika     8
dtype: int64
38

s1=pd.Series([10,20,30],index=["a","b","c"])
s2=pd.Series([1,2,3],index=["b","c","d"])
print(s1+s2)
#NaN returns float value
s=pd.Series([1,2,2,3,4,5,5])
print(s.count())
print(s.unique())
print(s.nunique())

a      NaN
b     21.0
c     32.0
d      NaN
dtype: float64
7
[1 2 3 4 5]
5

#accessing multiple elements
arr=pd.Series([10,20,30,40,50,60],index=["a","b","c","d","e","f"])
print(arr[["b","d","f"]])

b      20
d      40
f      60
dtype: int64

import pandas as pd
#method1 : from dictionary
data={
    "name":["monika","naghul","prem"],
    "rollno":[27,20,24],

```

```

    "Age": [19, 20, 21]}
df=pd.DataFrame(data)
print(df)
#method2 : from series to dataframe
s1=pd.Series([1,2,3,4,5])
s2=pd.Series([5.6,9.54,9.4,54.6,6])
s3=pd.Series([True,False,True,False])
df=pd.DataFrame({"s.no":s1,
                 "cgpa":s2,
                 "attendance":s3})

print(df)
print(df.loc[[1,2]])
#method3 : list of list
data=[
    ["athish", 22,98],
    ["rithika",20,89],
    ["prathiksha",20,97]]
df=pd.DataFrame(data,columns=["name","age","mark"],index=[1,2,3])
print(df)
#method4 : list of dictionary
data=[
    {"name":"sakthi",
     "course":"data science",
     "grade":"A"},
    {"name":"madhu",
     "course":"AIML",
     "grade":"A+"},
    {"name":"raagavi",
     "course":"AIML",
     "grade":"A++"}]
df=pd.DataFrame(data,index=["std1","std2","std3"])
print(df)
#adding one extra columns
df["marks"]=[50,75,80]
print(df)
#adding one extra row
df.loc["std4"]=["abi","CSDA","B","92"]
std5=pd.DataFrame([{"name":"alice","course":"bio","grade":"C","marks":
"55"}],index=["std5"])
df=pd.concat([df,std5])
print(df)
#create multiple rows

students=pd.DataFrame([{"name":"sanjeevi","course":"csda","grade":"C",
"marks":"55"},

{"name":"dhivya","course":"AIML","grade":"C","marks":"55"}],index=["st
d6","std7"])

df=pd.concat([df,students])

```

```

print(df)
#accessing particular column by bracket notation and dot notation
print(df.course) #only single column
print(df[['name','grade']]) #multiple column
#shape attribute
print(df.shape)
#find no of rows
print(df.shape[0])
#find no of columns
print(df.shape[1])
#dtype attribute
#used to check datatype of the columns
print(df.dtypes)
#info
print(df.info())
print(df.describe())
#select specific row
print(df.loc[["std1","std4"]])
print(df.iloc[1])
#update value
df.loc["std3","grade"]="B++"
print(df)
#delete column
df.drop("grade",axis=1,inplace=True)
print(df)
#delete row
df.drop("std6",axis=0,inplace=True)
print(df)
#sorting
df.sort_values(columns='marks',inplace=True)
print(df)
df.sort_values("marks",ascending=False,inplace=True)
print(df)
#statistical function for particular column
print(df["marks"].max())

```

	name	rollno	Age
0	monika	27	19
1	naghul	20	20
2	prem	24	21

	s.no	cgpa	attendance
0	1	5.60	True
1	2	9.54	False
2	3	9.40	True
3	4	54.60	False
4	5	6.00	NaN

	s.no	cgpa	attendance
1	2	9.54	False
2	3	9.40	True

	name	age	mark
--	------	-----	------

```

1      athish      22      98
2      rithika     20      89
3      prathiksha  20      97
      name          course grade
std1    sakthi     data science  A
std2    madhu          AIML    A+
std3    raagavi          AIML    A++
      name          course grade marks
std1    sakthi     data science  A      50
std2    madhu          AIML    A+      75
std3    raagavi          AIML    A++    80
      name          course grade marks
std1    sakthi     data science  A      50
std2    madhu          AIML    A+      75
std3    raagavi          AIML    A++    80
std4    abi          CSDA      B      92
std5    alice          bio      C      55
      name          course grade marks
std1    sakthi     data science  A      50
std2    madhu          AIML    A+      75
std3    raagavi          AIML    A++    80
std4    abi          CSDA      B      92
std5    alice          bio      C      55
std6    sanjeevi          csda      C      55
std7    dhivya          AIML      C      55
std1    data science
std2          AIML
std3          AIML
std4          CSDA
std5          bio
std6          csda
std7          AIML
Name: course, dtype: object
      name grade
std1    sakthi  A
std2    madhu   A+
std3    raagavi A++
std4    abi     B
std5    alice   C
std6    sanjeevi C
std7    dhivya  C
(7, 4)
7
4
name      object
course    object
grade     object
marks     object
dtype: object

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 7 entries, std1 to std7
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	name	7 non-null	object
1	course	7 non-null	object
2	grade	7 non-null	object
3	marks	7 non-null	object

```
dtypes: object(4)
```

```
memory usage: 280.0+ bytes
```

```
None
```

	name	course	grade	marks
count	7	7	7	7
unique	7	5	5	5
top	sakthi	AIML	C	55
freq	1	3	3	3

	name	course	grade	marks
std1	sakthi	data science	A	50
std4	abi	CSDA	B	92
name	madhu			
course	AIML			
grade	A+			
marks	75			

```
Name: std2, dtype: object
```

	name	course	grade	marks
std1	sakthi	data science	A	50
std2	madhu	AIML	A+	75
std3	raagavi	AIML	B++	80
std4	abi	CSDA	B	92
std5	alice	bio	C	55
std6	sanjeevi	csda	C	55
std7	dhivya	AIML	C	55

	name	course	marks
std1	sakthi	data science	50
std2	madhu	AIML	75
std3	raagavi	AIML	80
std4	abi	CSDA	92
std5	alice	bio	55
std6	sanjeevi	csda	55
std7	dhivya	AIML	55

	name	course	marks
std1	sakthi	data science	50
std2	madhu	AIML	75
std3	raagavi	AIML	80
std4	abi	CSDA	92
std5	alice	bio	55
std7	dhivya	AIML	55


```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[23], line 81
```

```
    79 print(df)  
    80 #sorting  
--> 81 df.sort_values('marks',inplace=True)  
    82 print(df)  
    83 df.sort_values("marks",ascending=False,inplace=True)
```

```
File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:7200, in  
DataFrame.sort_values(self, by, axis, ascending, inplace, kind,  
na_position, ignore_index, key)
```

```
    7197     if isinstance(ascending, (tuple, list)):  
    7198         ascending = ascending[0]  
-> 7200     indexer = nargsort(  
    7201         k, kind=kind, ascending=ascending,  
na_position=na_position, key=key  
    7202     )  
    7203 else:  
    7204     if inplace:
```

```
File ~\anaconda3\Lib\site-packages\pandas\core\sorting.py:439, in  
nargsort(items, kind, ascending, na_position, key, mask)
```

```
    437     non_nans = non_nans[::-1]  
    438     non_nan_idx = non_nan_idx[::-1]  
-> 439     indexer = non_nan_idx[non_nans.argsort(kind=kind)]  
    440     if not ascending:  
    441         indexer = indexer[::-1]
```

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

```
import pandas as pd  
#read csv files #usecols if you need specific column  
df=pd.read_csv("people-100.csv",usecols=["Last Name","Sex"])  
print(df)  
#read only 1st 5 data  
print(df.head())  
#read last 5 data  
print(df.tail())  
print(df.to_string())  
print(df["Sex"])
```

	Last Name	Sex
0	Terrell	Male
1	Summers	Female
2	Travis	Male
3	Martinez	Male
4	Todd	Male

95	Barnes	Female
96	Patterson	Female
97	Bray	Male
98	Oconnell	Female
99	Bernard	Male

[100 rows x 2 columns]

	Last Name	Sex
0	Terrell	Male
1	Summers	Female
2	Travis	Male
3	Martinez	Male
4	Todd	Male
	Last Name	Sex
95	Barnes	Female
96	Patterson	Female
97	Bray	Male
98	Oconnell	Female
99	Bernard	Male
	Last Name	Sex
0	Terrell	Male
1	Summers	Female
2	Travis	Male
3	Martinez	Male
4	Todd	Male
5	Day	Male
6	Buck	Female
7	Hinton	Male
8	Farrell	Male
9	Downs	Male
10	Ross	Female
11	Newton	Male
12	Blake	Male
13	Nash	Female
14	Farmer	Male
15	Knight	Female
16	Rangel	Male
17	Meadows	Male
18	Paul	Male
19	Mitchell	Female
20	Norton	Female
21	Hatfield	Female
22	Barnes	Male
23	Rhodes	Female
24	Mcneil	Female
25	Levine	Male
26	Payne	Male
27	Vega	Female

28	Buckley	Male
29	Weber	Male
30	Camacho	Female
31	Gallagher	Male
32	Andrews	Female
33	Schmidt	Female
34	Sparks	Female
35	Cook	Male
36	Hurst	Male
37	Rich	Female
38	Wiggins	Female
39	Mckinney	Female
40	Stone	Female
41	Walsh	Female
42	Vang	Female
43	Hill	Female
44	Andrews	Male
45	Pugh	Female
46	Estrada	Female
47	Brady	Male
48	Frey	Male
49	Sweeney	Male
50	Escobar	Female
51	Oconnell	Female
52	Esparza	Female
53	Rivas	Female
54	Payne	Female
55	Rocha	Female
56	Watkins	Female
57	Olsen	Female
58	Nelson	Female
59	Phelps	Male
60	Daugherty	Male
61	Garner	Male
62	Bennett	Male
63	Conway	Female
64	Barnett	Female
65	Harper	Male
66	Ward	Female
67	Rubio	Female
68	Pineda	Male
69	Wu	Male
70	Benton	Male
71	Hull	Male
72	Ritter	Female
73	Briggs	Female
74	Andrews	Female
75	Cunningham	Male
76	Atkinson	Male

77	Reid	Female
78	Stevens	Male
79	Huff	Male
80	Foley	Female
81	Waters	Male
82	Stephenson	Male
83	Reese	Female
84	Williamson	Female
85	Vaughn	Male
86	Dickson	Male
87	Bailey	Male
88	Medina	Female
89	Haney	Female
90	Nixon	Female
91	Hardy	Female
92	Cox	Female
93	Cole	Male
94	Guzman	Female
95	Barnes	Female
96	Patterson	Female
97	Bray	Male
98	Oconnell	Female
99	Bernard	Male
0	Male	
1	Female	
2	Male	
3	Male	
4	Male	
	...	
95	Female	
96	Female	
97	Male	
98	Female	
99	Male	

Name: Sex, Length: 100, dtype: object

Day4

```
import pandas as pd
df=pd.read_json("titanic.json")
print(df.head(10))
df.tail()
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	